USE CASE DIAGRAM

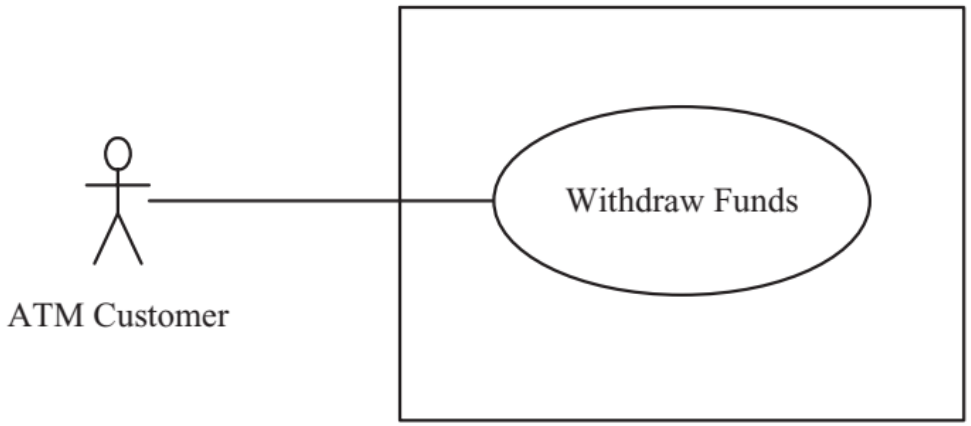
CICT - CTU

Outline

- Use case diagram
  - Use Case
  - Actor
  - Identifying use cases
  - Documenting use cases
  - Use case relationships
- Tool: PowerDesigner

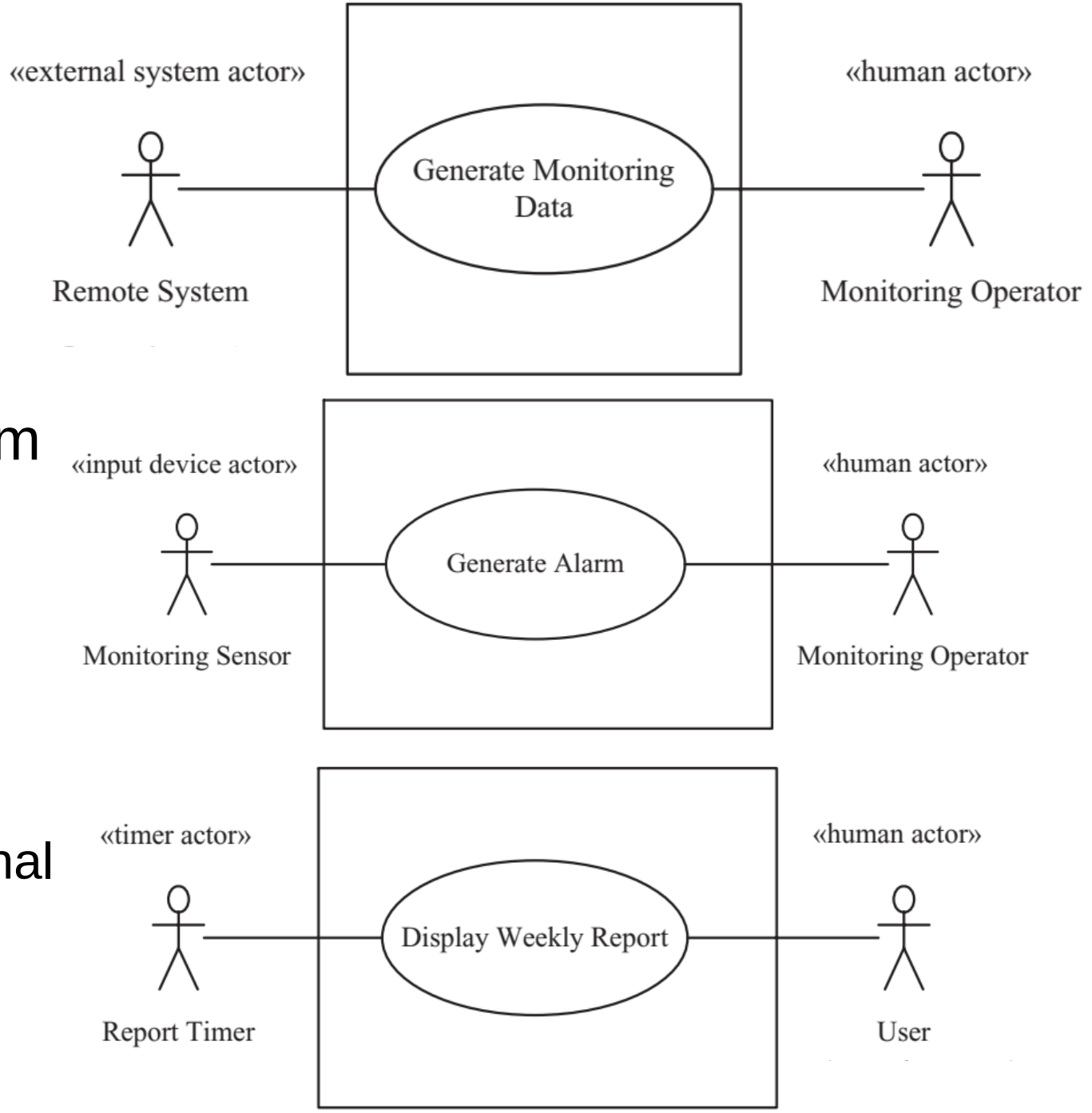
Use Case

- Use Case model
  - Use case model define system functional requirements in terms of Actors and Use cases.
- Use Case
  - Use case describes sequence of interactions between one or more actors and the system.
  - A use case always starts with input from an actor.
  - Each interaction consists of an input from the actor followed by a response from the system.



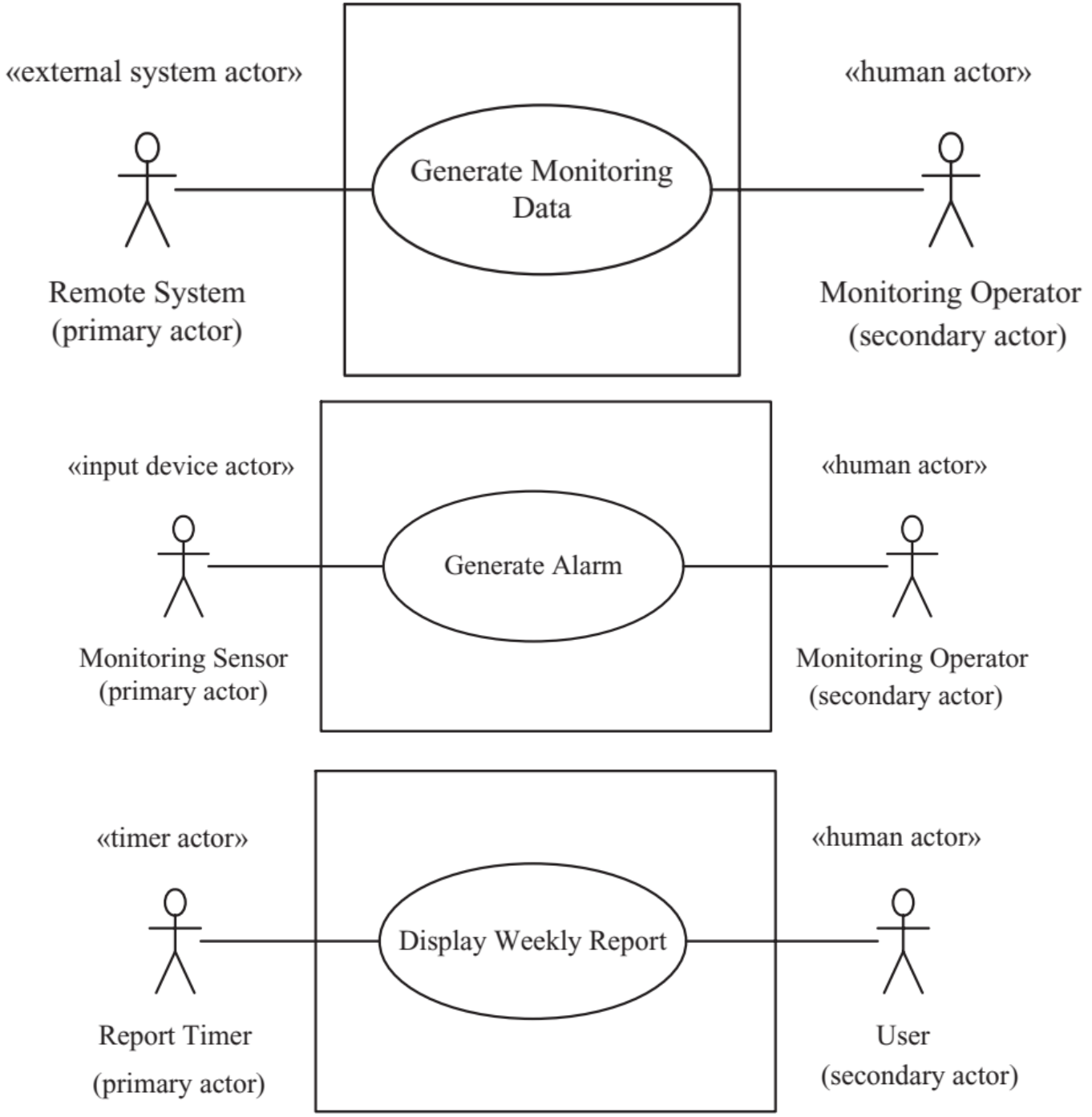
Actor

- Actor models external entities of system - outside the system and not part of it
- Actors interact directly with system
  - Human user
  - External I/O device
  - Timer
  - External system
- Actor initiates actions by system
  - Actor may use I/O devices or external system to physically interact with system
  - Actor initiates use cases



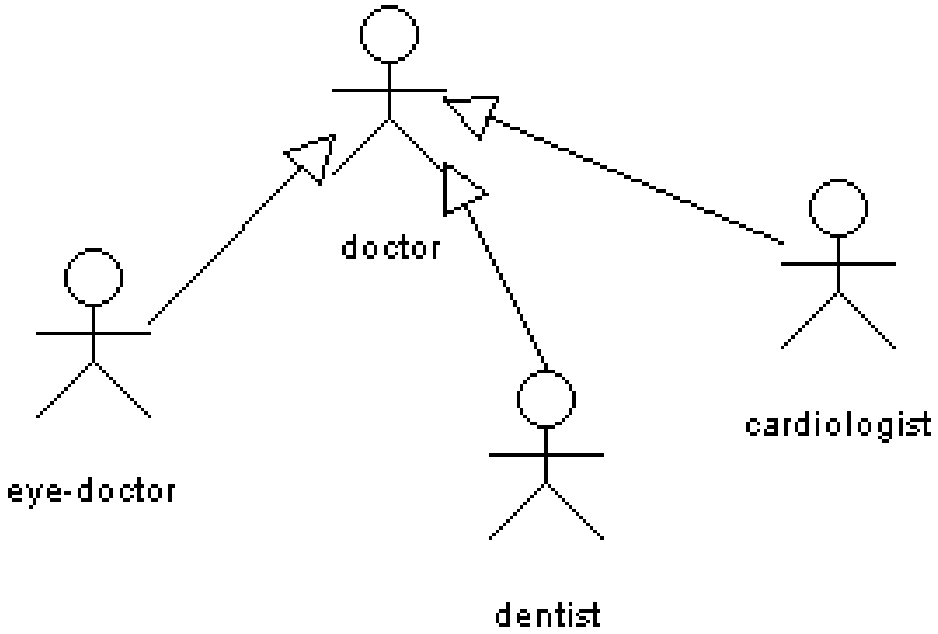
Actor

- Primary Actor starts the use case by providing input to the system.
- Secondary Actor participates in use case, can be Primary Actor of a different use case.
- Actor
  - Represents all users who use system in the same way
    - A user is an instance of an actor
  - Represents a role played by all users of the same type
    - Human user may play more than one role



Actor

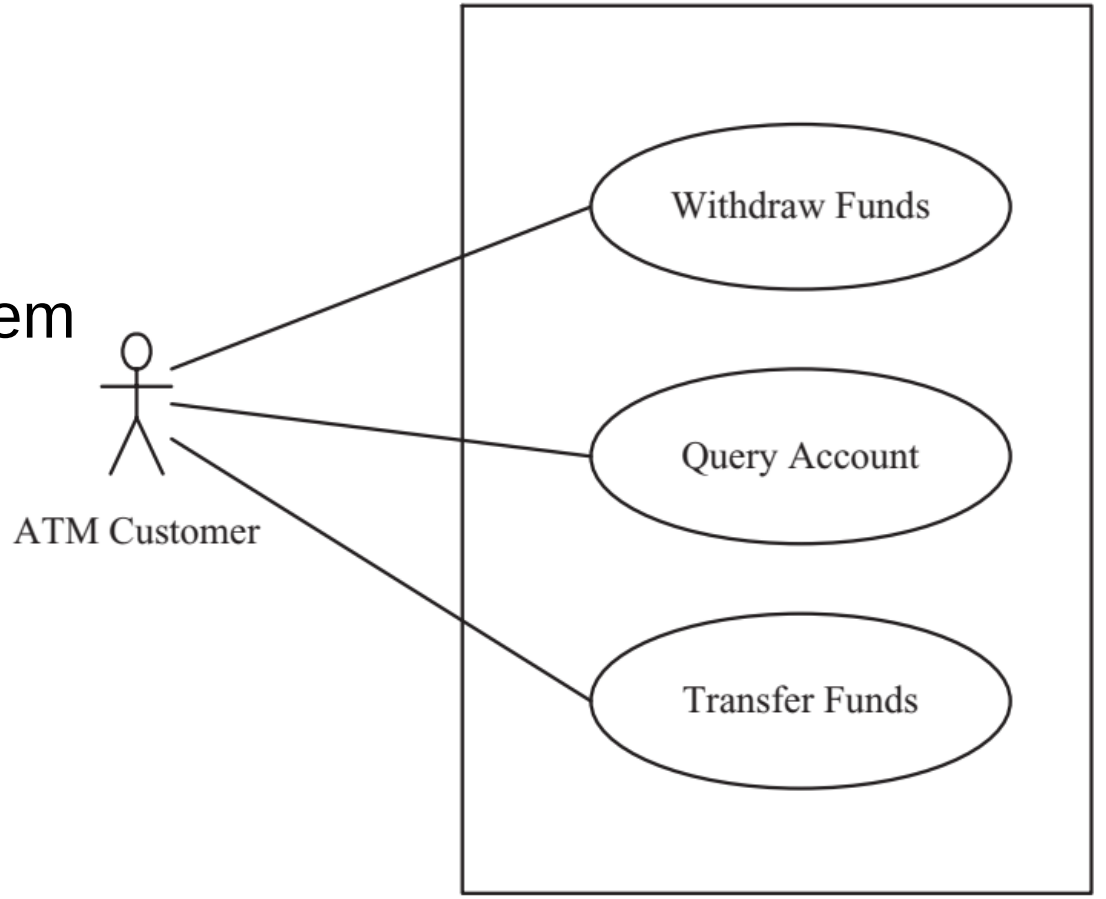
- Actors can participate in a generalization relation with other actors.



- Who or what will use the main functionality of the system?
- Who or what will provide input to this system?
- Who or what will use output from this system?
- Who will need support from the system to do their work?
- Are there any other software systems with which this one needs to interact
- Are there any hardware devices used or controlled by this system?

Use Case - Identifying use cases

- Consider each major function an actor needs to perform
- Provides value to actor
- Use case is a complete sequence of events initiated by an actor
  - Specifies interaction between actor and system
- Use case starts with input from an actor
- Basic path (normal flow)
  - Most common sequence
- Alternative branches (alternative flows)
  - Variants of basic path that are executed only under certain circumstances. E.g., for error handling



Use Case - Documenting

- Name
- Summary: short description of use case
- Dependency (on other use cases): describes whether the use case depends on other use cases (includes or extends)
- Actors
- Preconditions: Conditions that are true at start of use case
- Description: Narrative description of basic path
- Alternatives: Narrative description of alternative paths
- Postcondition: Condition that is true at end of use case

Use Case - Documenting - Example of Use Case

- **Use Case Name:** Withdraw Funds
- **Summary:** Customer withdraws a specific amount of funds from a valid bank account.
- **Actor:** ATM Customer
- **Precondition:** ATM is idle, displaying a Welcome message.
- **Description:**

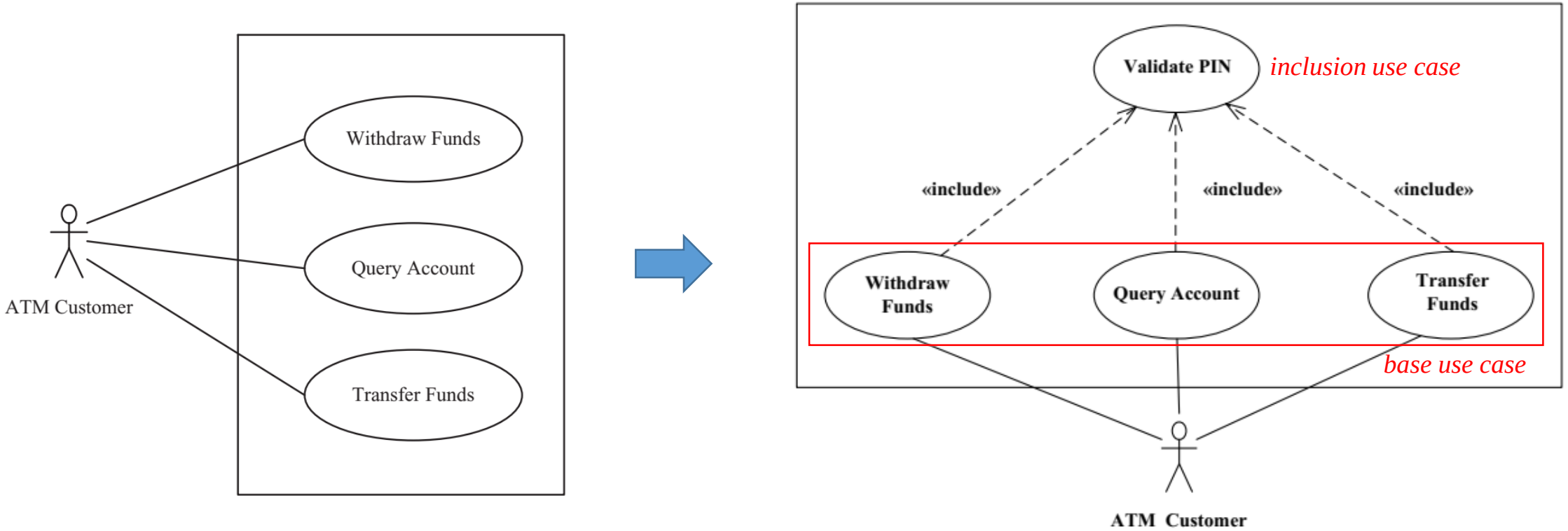
1. Customer inserts the ATM Card into the Card Reader.
2. If the system recognizes the card, it reads the card number.
3. System prompts customer for PIN number.
4. Customer enters PIN.
5. System checks the expiration date and whether the card is lost or stolen.
6. If card is valid, the system then checks whether the user-entered PIN matches the card PIN maintained by the system.
7. If PIN numbers match, the system checks what accounts are accessible with the ATM Card.
8. System displays customer accounts and prompts customer for transaction type: Withdrawal, Query, or Transfer.
9. Customer selects Withdrawal, enters the amount, and selects the account number.
10. System checks whether customer has enough funds in the account and whether daily limit has been exceeded.
11. If all checks are successful, system authorizes dispensing of cash.
12. System dispenses the cash amount.
13. System prints a receipt showing transaction number, transaction type, amount withdrawn, and account balance.
14. System ejects card.
15. System displays Welcome message

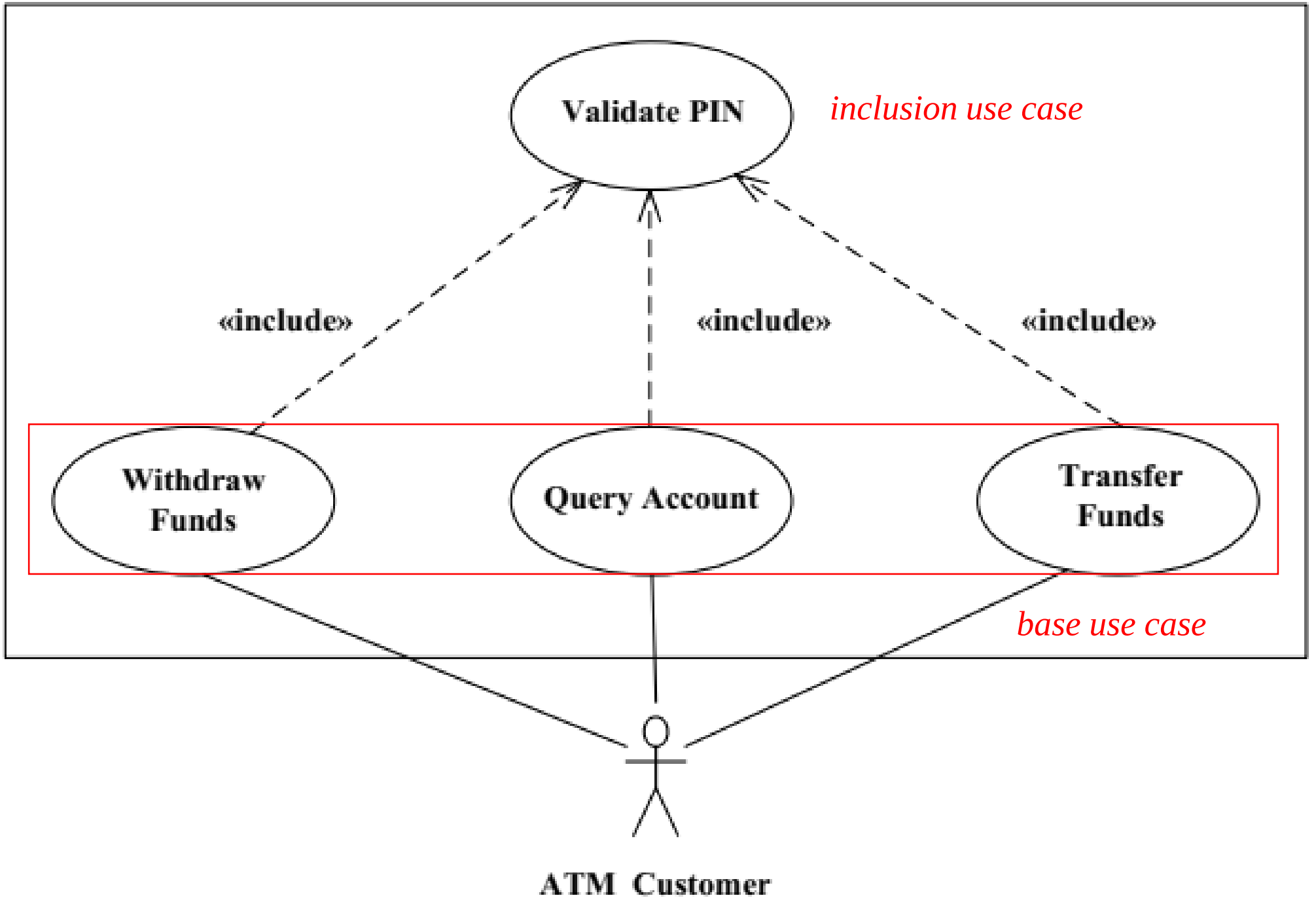
Use Case - Documenting - Example of Use Case

- **Alternatives:**
  - If the system does not recognize the card, the card is ejected.
  - If the system determines that the card date has expired, the card is confiscated.
  - If the system determines that the card has been reported lost or stolen, the card is confiscated.
  - If the customer entered PIN does not match the PIN number for this card, then the system re-prompts for the PIN.
  - If the customer enters the incorrect PIN three times, then the system confiscates the card.
  - If the system determines that the account number is invalid, then it displays an error message and ejects the card.
  - If the system determines that there are insufficient funds in the customer's account, then it displays an apology and ejects the card.
  - If the system determines that the maximum allowable daily withdrawal amount has been exceeded, then it displays an apology and ejects the card.
  - If the ATM is out of funds, then the system displays an apology, ejects the card, and shuts down the ATM.
  - If the customer enters Cancel, the system cancels the transaction and ejects the card.
- **Postcondition:** Customer funds have been withdrawn.

Use Case – Relationships - Include

- **Include** relationship identifies common sequences of interactions in several use cases
  - Extract common sequence into **inclusion use case**
  - Base use cases **include** abstract use case
  - Example: Withdraw Funds use case **includes** Validate PIN use case




Validate PIN
inclusion use case
«include»
«include»
«include»
Withdraw Funds
Query Account
Transfer Funds
base use case
ATM Customer

Use Case - Documenting - Example of Use Case

- **Use Case Name:** Withdraw Funds
- **Summary:** Customer withdraws a specific amount of funds from a valid bank account.
- **Actor:** ATM Customer
- **Precondition:** ATM is idle, displaying a Welcome message.
- **Description:**

1. Customer inserts the ATM Card into the Card Reader.
2. If the system recognizes the card, it reads the card number.
3. System prompts customer for PIN number.
4. Customer enters PIN.
5. System checks the expiration date and whether the card is lost or stolen.
6. If card is valid, the system then checks whether the user-entered PIN matches the card PIN maintained by the system.
7. If PIN numbers match, the system checks what accounts are accessible with the ATM Card.
8. System displays customer accounts and prompts customer for transaction type: Withdrawal, Query, or Transfer.
9. Customer selects Withdrawal, enters the amount, and selects the account number.
10. System checks whether customer has enough funds in the account and whether daily limit has been exceeded.
11. If all checks are successful, system authorizes dispensing of cash.
12. System dispenses the cash amount.
13. System prints a receipt showing transaction number, transaction type, amount withdrawn, and account balance.
14. System ejects card.
15. System displays Welcome message

Use Case - Documenting - Example of Use Case

- **Alternatives:**
  - If the system does not recognize the card, the card is ejected.
  - If the system determines that the card date has expired, the card is confiscated.
  - If the system determines that the card has been reported lost or stolen, the card is confiscated.
  - If the customer entered PIN does not match the PIN number for this card, then the system re-prompts for the PIN.
  - If the customer enters the incorrect PIN three times, then the system confiscates the card.
  - If the system determines that the account number is invalid, then it displays an error message and ejects the card.
  - If the system determines that there are insufficient funds in the customer's account, then it displays an apology and ejects the card.
  - If the system determines that the maximum allowable daily withdrawal amount has been exceeded, then it displays an apology and ejects the card.
  - If the ATM is out of funds, then the system displays an apology, ejects the card, and shuts down the ATM.
  - If the customer enters Cancel, the system cancels the transaction and ejects the card.
- **Postcondition:** Customer funds have been withdrawn.

Use Case - Relationship - Example of Inclusion Use Case

- **Use Case Name:** Validate PIN
- **Summary:** System validates customer PIN.
- **Actor:** ATM Customer
- **Precondition:** ATM is idle, displaying a Welcome message.
- **Description:**
  1. Customer inserts the ATM Card into the Card Reader.
  2. If the system recognizes the card, it reads the card number.
  3. System prompts customer for PIN number.
  4. Customer enters PIN.
  5. System checks the expiration date and whether the card is lost or stolen.
  6. If card is valid, the system then checks whether the user-entered PIN matches the card PIN maintained by the system.
  7. If PIN numbers match, the system checks what accounts are accessible with the ATM Card.
  8. System displays customer accounts and prompts customer for transaction type: Withdrawal, Query, or Transfer.
- **Alternatives**:
  - If the system does not recognize the card, the card is ejected.
  - If the system determines that the card date has expired, the card is confiscated.
  - If the system determines that the card has been reported lost or stolen, the card is confiscated.
  - If the customer-entered PIN does not match the PIN number for this card, the system re-prompts for PIN.
  - If the customer enters the incorrect PIN three times, the system confiscates the card.
  - If the customer enters Cancel, the system cancels the transaction and ejects the card.

Postcondition: Customer PIN has been validated.

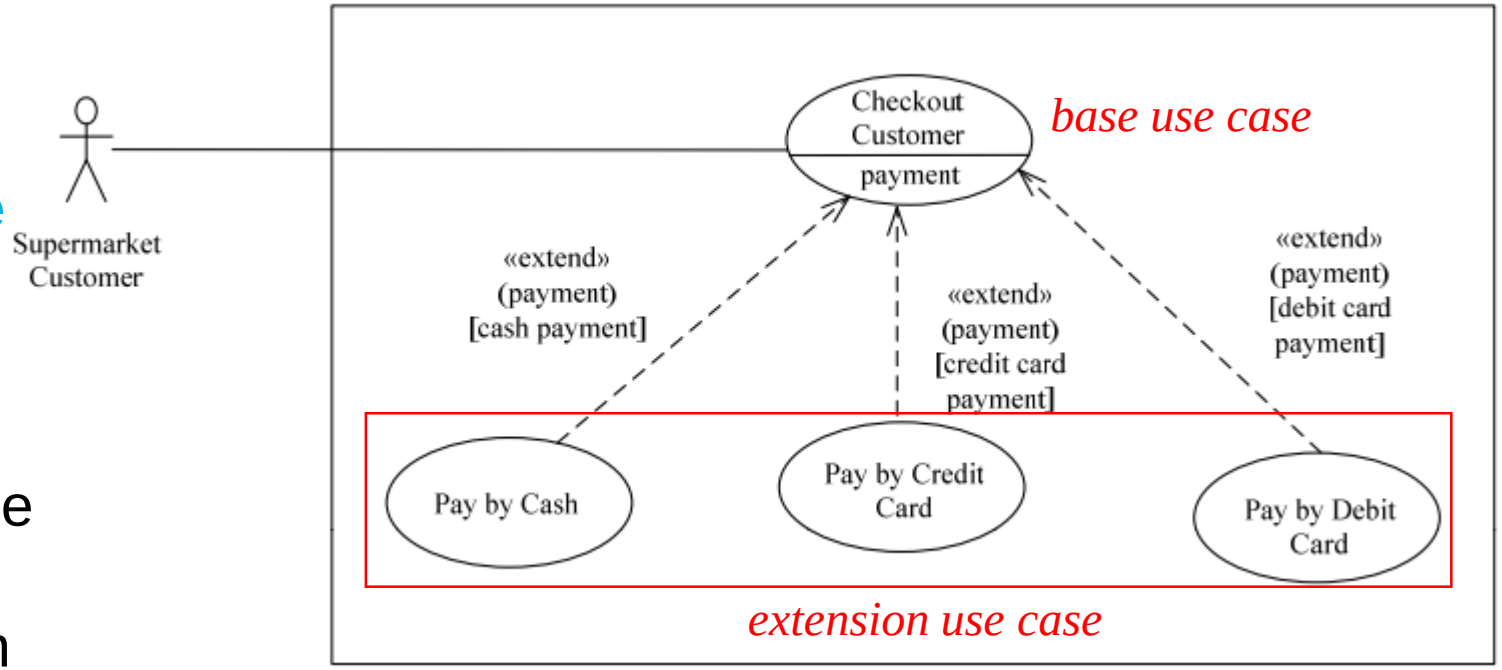
Use Case - Relationship - Example of Base Use Case

- **Use Case Name:** Withdraw Funds
- **Summary:** Customer withdraws a specific amount of funds from a valid bank account.
- **Actor:** ATM Customer
- **Precondition:** ATM is idle, displaying a Welcome message.
- **Description:**

  1. Include Validate PIN abstract use case.
  2. Customer selects Withdrawal, enters the amount, and selects the account number.
  3. System checks whether customer has enough funds in the account and whether daily limit has been exceeded.
  4. If all checks are successful, system authorizes dispensing of cash.
  5. System dispenses the cash amount.
  6. System prints a receipt showing transaction number, transaction type, amount withdrawn, and account balance.
  7. System ejects card.
  8. System displays Welcome message
- **Alternatives:**
  - If the system determines that the account number is invalid, then it displays an error message and ejects the card.
  - If the system determines that there are insufficient funds in the customer's account, then it displays an apology and ejects the card.
  - If the system determines that the maximum allowable daily withdrawal amount has been exceeded, then it displays an apology and ejects the card.
  - If the ATM is out of funds, then the system displays an apology, ejects the card, and shuts down the ATM.
- **Postcondition:** Customer funds have been withdrawn.

Use Case – Relationships - Extend

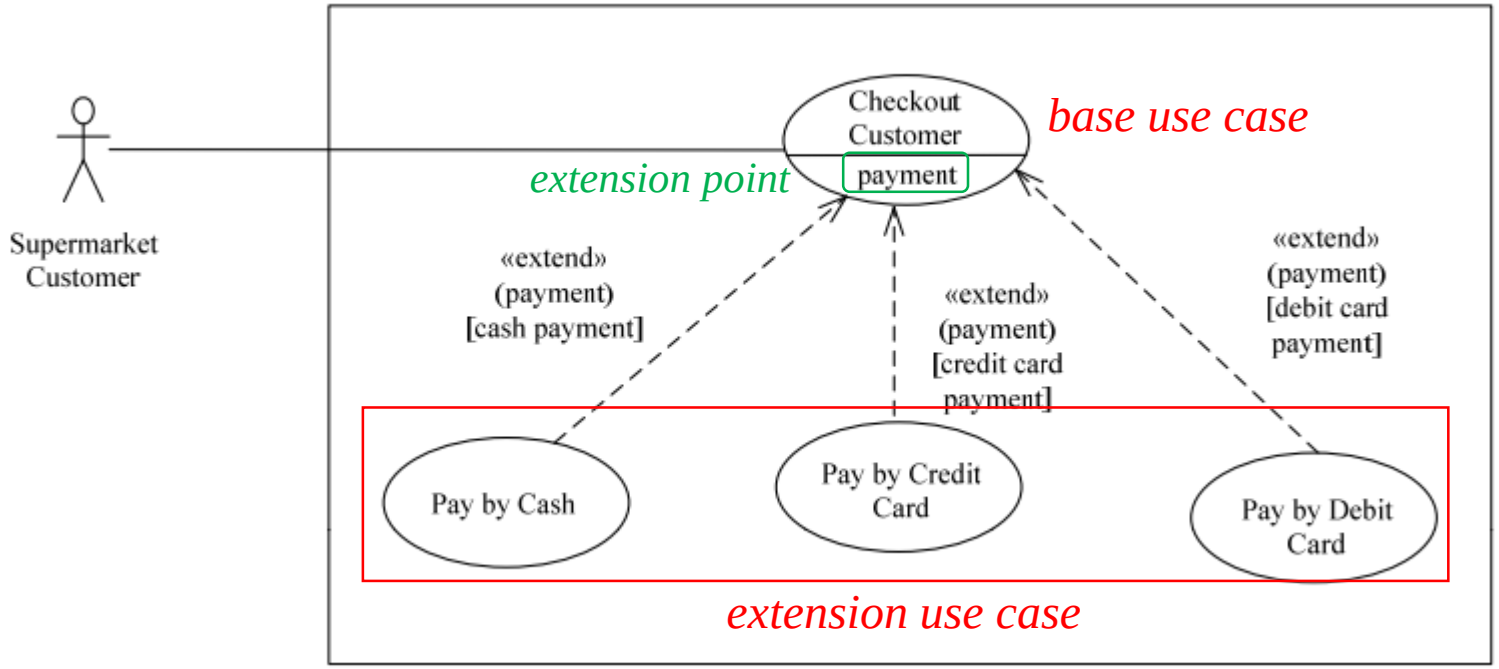
- **Extend** relationship
  - The use case that is extended is referred to as the **base use case**
  - The use case that does the extending is referred to as the **extension use case**
  - Under certain conditions, the base use case will be extended by description given in the extension use case
  - Same base use case can be extended in different ways



- When to use **extend**
  - To show a conditional part of the base use case that is executed only under certain circumstances
  - To model complex or alternative paths.

Use Case – Relationships - Extend

- **Extension point**
  - Extension points are used to specify the precise locations in the base use case at which extensions can be added
  - An extension use case may extend the base use case only at these extension points
  - It is possible to have more than one extension use case for the same extension point, but with each extension use case satisfying a different condition.



- The extension conditions are designed such that only one condition can be true and, hence, only one extension use case selected, for any given situation.

Use Case – Relationships - Extend

Checkout Customer Base Use Case

- **Use case name:** Checkout Customer
- **Summary:** System checks out customer.
- **Actor:** Customer
- **Precondition:** Checkout station is idle, displaying a “Welcome” message.
- **Main sequence:**
  1. Customer scans selected item.
  2. System displays the item name, price, and cumulative total.
  3. Customer repeats steps 1 and 2 for each item being purchased.
  4. Customer selects payment.
  5. System prompts for payment by cash, credit card, or debit card.
  6. «payment»
  7. System displays thank-you screen.

Pay by Cash Extension Use Case

- **Use case name:** Pay by Cash
- **Summary:** Customer pays by cash for items purchased.
- **Actor:** Customer
- **Dependency:** Extends Checkout Customer.
- **Precondition:** Customer has scanned items but not yet paid for them.
- **Description of insertion segment:**
  1. Customer selects payment by cash.
  2. System prompts customer to deposit cash in bills and/or coins.
  3. Customer enters cash amount.
  4. System computes change.
  5. System displays total amount due, cash payment, and change.
  6. System prints total amount due, cash payment, and change on receipt.

Use Case – Relationships - Extend

Pay by Cash Extension Use Case

- **Use case name:** Pay by Cash
- **Summary:** Customer pays by cash for items purchased.
- **Actor:** Customer
- **Dependency:** Extends Checkout Customer.
- **Precondition:** Customer has scanned items but not yet paid for them.
- **Description of insertion segment:**
  1. *Customer selects payment by cash.*
  2. System prompts customer to deposit cash in bills and/or coins.
  3. Customer enters cash amount.
  4. System computes change.
  5. System displays total amount due, cash payment, and change.
  6. System prints total amount due, cash payment, and change on receipt.

Pay by Credit Card Extension Use Case

- **Use case name:** Pay by Credit Card
- **Summary:** Customer pays by credit card for items purchased.
- **Actor:** Customer
- **Dependency:** Extends Checkout Customer.
- **Precondition:** Customer has scanned items but not yet paid for them.
- **Description of insertion segment:**
  1. *Customer selects payment by credit card.*
  2. System prompts customer to swipe card.
  3. Customer swipes card.
  4. System reads card ID and expiration date.
  5. System sends transaction to authorization center containing card ID, expiration date, and payment amount.
  6. If transaction is approved, authorization center returns positive confirmation.
  7. System displays payment amount and confirmation.
  8. System prints payment amount and confirmation on receipt.

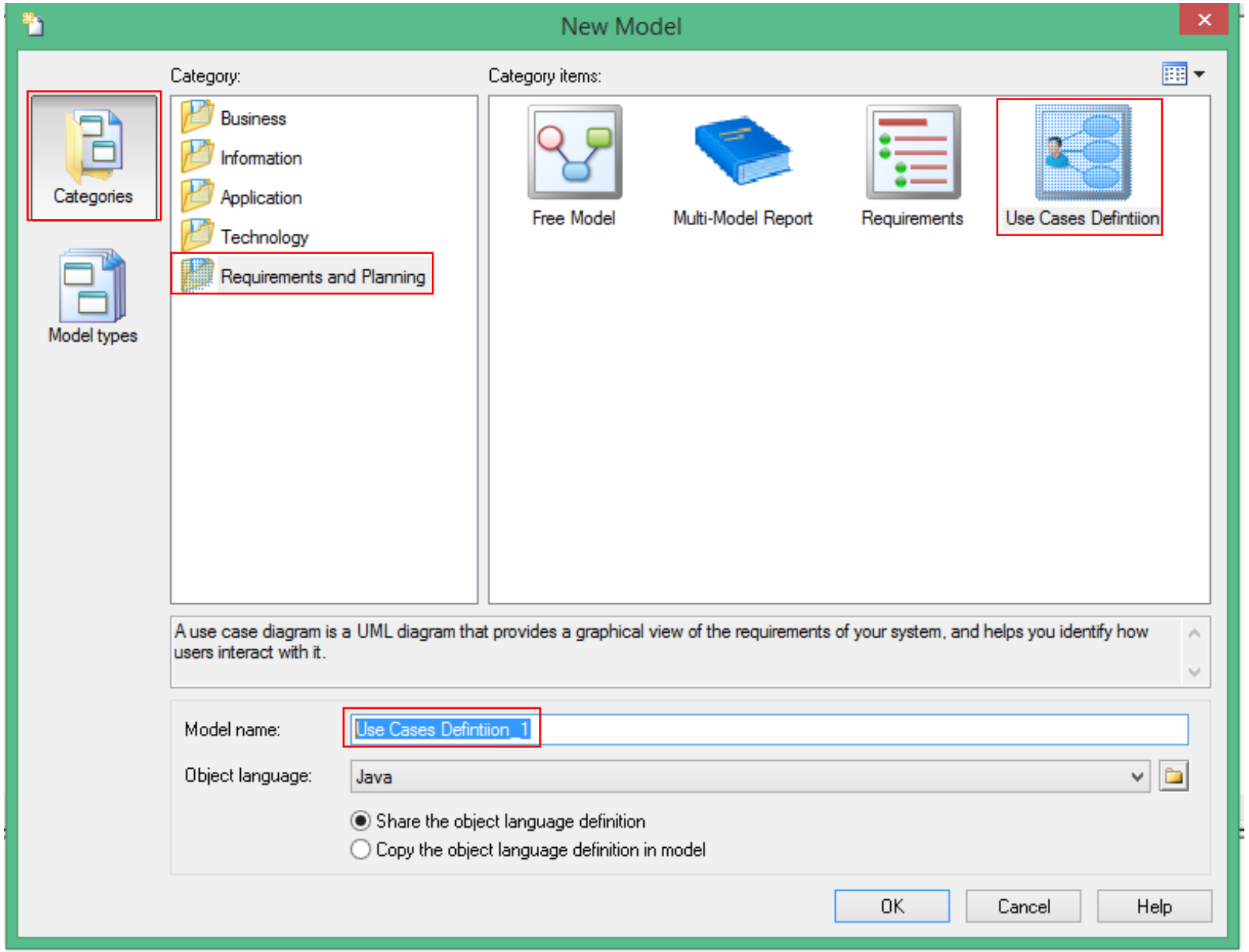
PowerDesigner

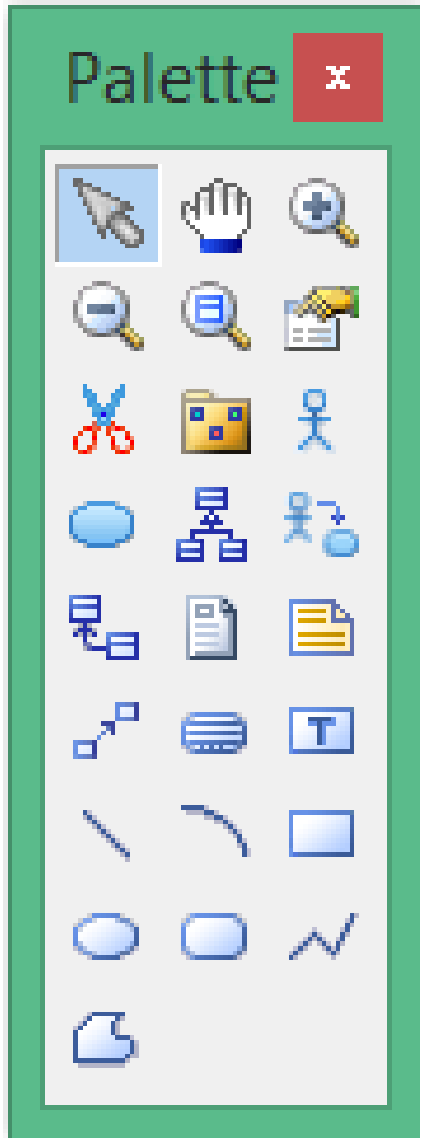
- Object-Oriented Architecture SAP PowerDesigner Documentation Collection
  - 1.2 Use Case Diagrams: Page 20
  - 1.3.14 Dependencies (OOM): Page 97
  - 1.3.13 Generalizations (OOM): Page 94

  https://infocenter-archive.sybase.com/help/index.jsp?topic=/com.sybase.infocenter.dc38086.1530/doc/html/rad1232632517798.html

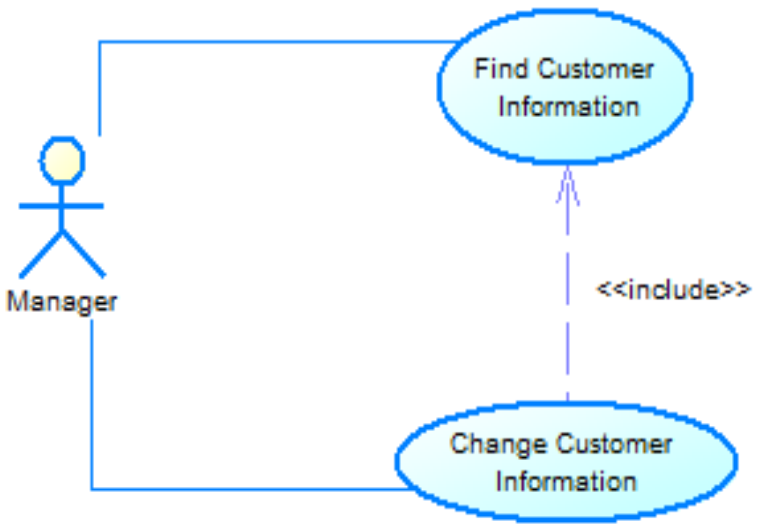
- File
- New Model

PowerDesigner





PowerDesigner



Use Case: Change Customer Information	**ID:** UC001
Main actor: Manager	**Priority:** Essential

Brief description: *A valid manager logged on to the system.* She/he wants to change some information of customer.

Trigger: Manager **Type:** external

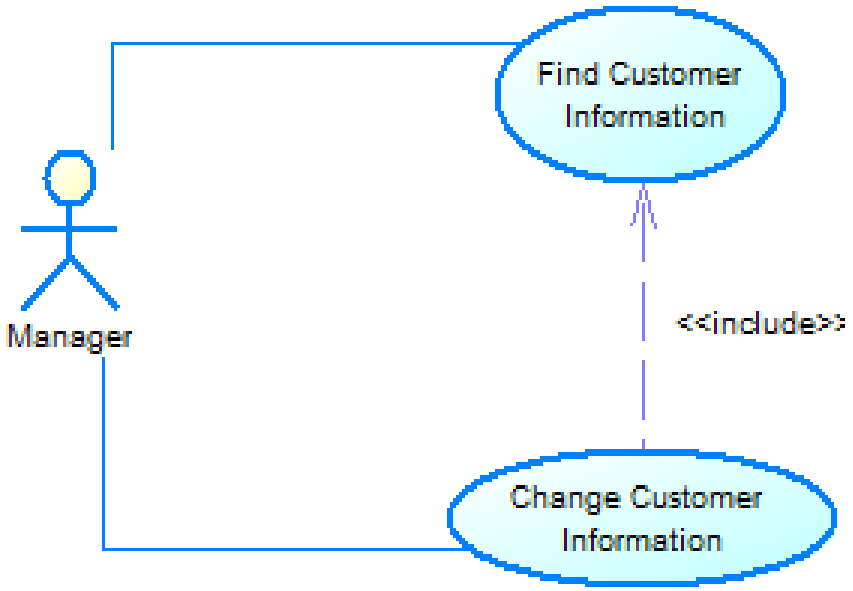
Relationship:
+Association: Manager – Change Customer Information
+Include: Find Customer Information
+Extend: None **+Generalization:** None

Normal flow:

1. Include "Find Customer Information".
2. The Manager selects the part of customer information to change.
3. The Manager update the information of selected path and requests that the system saves the entered values.
4. The system validates the entered customer information.
   4.1. The system describes which entered data was invalid and presents the Manager with suggestions for entering valid data.
   4.2. The system prompts the Manager to re-enter the invalid information.
   4.3. The Manager re-enters the information and the system re-validates it.
   4.4. If valid information is entered, go to Step 5; else go to Step 4.1 or cancel the change customer information request. If the Manager entered invalid data or chose to cancel the change request, there is no change to the Customer's record.
5. The updated customer information is stored in the Customer's record. The system notifies the Manager that the customer information has been updated.

Exceptional flows:
At any time, the Manager may choose to cancel the information update. At which point, the processing is discontinued, the Customer's record unchanged, and the Manager is notified that the request has been cancelled.

PowerDesigner



Use Case: Find Customer Information	**ID:** UC002
Main actor: Manager	**Priority:** Essential
Brief description: *A valid manager logged on to the system.* She/he wants to find the information of customer.	
Trigger: Manager **Type:** external	
Relationship: **+Association:** Manager – Find Customer Information **+Include:** None **+Extend:** None **+Generalization:** None	
Normal flow: 1. The Manager enters the ID number of customer. 2. The system finds the information of customer. 3. The system displays those information on a form.	

References

- Hassan Gomaa, *Software Modeling and Design: UML, Use Cases, Patterns, and Software Architectures*, Cambridge University Press, Illustrated edition, 2011.
- SAP SE, *Object-Oriented Architecture SAP PowerDesigner Documentation Collection*, 2018.

Use Case Specification Template

Number	*Unique use case number*	
Name	*Brief noun-verb phrase*	
Summary	*Brief summary of use case major actions*	
Priority	*1-5 (1 = lowest priority, 5 = highest priority)*	
Preconditions	*What needs to be true before use case "executes"*	
Postconditions	*What will be true after the use case successfully "executes"*	
Primary Actor(s)	*Primary actor name(s)*	
Secondary Actor(s)	*Secondary actor name(s)*	
Trigger	*The action that causes this use case to begin*	
Main Scenario	**Step**	**Action**
	Step #	*This is the "main success scenario" or "happy path."*
	…	*Description of steps in successful use case* "execution"
	…	*This should be in a "system-user-system, etc." format.*
Extensions	**Step**	**Branching Action**
	Step #	*Alternative paths that the use case may take*
Open Issues	*Issue #*	*Issues regarding the use case that need resolution*

Extension

- Could be an optional path(s)
- Could be an error path(s)
- Denoted in use case diagrams (UML) by **<<extend>>**